

Read-Only Is Not Wear-Free: The Reliability Wall of High-Bandwidth Flash for LLM Inference

Garima Kalra

Department of Computer Science and Engineering
Indian Institute of Technology Guwahati
k.garima@iitg.ac.in

Phrangboklang Lyngton Thangkhiew

Department of Computer Science and Engineering
Indian Institute of Technology Guwahati

Abstract—High-Bandwidth Flash (HBF) is being proposed as the capacity tier that lets a single GPU hold what today takes dozens: model weights and shared KV caches that are read on every decode step but almost never written. Every published HBF design, more than twenty in 2026 alone, rests on the same premise: because this data is read-only, NAND’s weak endurance does not matter. We argue that the premise is wrong in exactly the regime HBF is built for. Data that is read at a terabyte per second, next to a kilowatt GPU, for years, is the worst case for three NAND failure mechanisms that no HBF evaluation models together: read disturb, retention loss at high temperature, and the error-correction and read-retry cost of keeping data correct in between. The HBF base-die specification leaves refresh for data age and read counts to the host, and no host-side design exists.

Using an analytical model calibrated on the H³ architecture, we show that each static block of a ten-million-token Llama-3.1-405B deployment is read about 5×10^5 times per day, two to six orders of magnitude beyond published read-disturb limits over a five-year service life. Read reclaim must rewrite the “read-only” tier every 11–28 minutes, which consumes the device in 2–5 years without static wear levelling, and the retention window itself shrinks from 24 hours to about 4 hours at a 105 °C junction. Two properties of LLM inference make this tractable: the access schedule is known in advance, and most HBF capacity is idle, because the flash tier is over-provisioned 17–67× for eight of nine models. We outline Steward, a base-die reliability manager that exploits both, using schedule-derived read accounting, replica rotation in spare capacity and criticality-aware protection, and we set out the characterisation and evaluation that would establish whether HBF can serve LLMs reliably at all.

Index Terms—high-bandwidth flash, NAND reliability, read disturb, data retention, LLM inference, heterogeneous memory

I. INTRODUCTION

Serving a large language model over a long shared context is limited by memory capacity. At ten million tokens, the weights and pre-computed KV cache of Llama-3.1-405B occupy 5.8 TB, the HBM of 31 B200-class GPUs, before a single request is admitted [1]. High-Bandwidth Flash stacks NAND dies behind a logic base die as HBM stacks DRAM, targeting HBM-class read bandwidth at up to 16× the capacity [20], [22]. A base-die specification appeared through the Open Compute Project in August 2026 [21], first parts are expected in 2027 [13], and the architecture literature has moved quickly: HBF has been chained behind HBM, placed beside it, used to replace it, and co-designed with MoE expert streaming,

sparse attention, agentic session tiering and recommendation serving [1]–[12], [17]–[19].

Nearly all of these designs share one argument. Flash is slow to write and wears out, so it should hold data that is read-only: weights, shared prefixes, immutable KV. H³ states that under read-only placement the weak endurance of HBF “is no longer a disadvantage” [1], and most later work inherits the conclusion. Where papers do analyse endurance, they count *writes*, and when they disagree it is about how many writes KV caching produces [4]–[6], [8], [10].

This paper argues that the read-only premise fails, and that it fails precisely because of how HBF is meant to be used. NAND wears through reads and through time, not only through programs. Every read of a page slightly disturbs the other wordlines of its block, and after enough reads the block must be rewritten (*read reclaim*) [39]. Stored charge leaks, faster at high temperature, so data must be rewritten within a retention window [40], [41]. Between rewrites the raw bit-error rate rises, which costs error-correction work and eventually read retries on the critical path [40]. These effects are mild for an SSD that reads a block a few times an hour at 40 °C. An HBF stack serving model weights re-reads every static block on every decode step, several times per second, sitting on an interposer next to a GPU that dissipates about a kilowatt. That is the worst case for all three mechanisms at once.

The problem has also been assigned, by specification, to computer architects. The OCP base-die specification leaves refresh for data age and read-disturb counts to the host, and bounds powered-on retention at 24 hours at 85 °C [12], [13], [21]. In an SSD this work is done by a controller with its own DRAM and firmware, hidden behind a block interface. With HBF it must be done by the HBM base die, the GPU memory controller or the runtime, at terabytes per second, without stalling decode. Of the proposals we surveyed, one models read-disturb refresh for an HBF weight tier, without temperature or error-correction cost [9]; one simulator implements retention and refresh but reports no results [13]; and none evaluates what raw bit errors do to model accuracy, what error correction costs at HBF bandwidth, or how any of it changes with the provisioning choices the rest of the literature is busy making (Section III).

Problem statement. *Can a flash tier keep terabytes of read-*

mostly LLM state correct, for a multi-year service life, next to a kilowatt GPU, without giving back the capacity and bandwidth that justified it? If so, what must the host side of the memory system do, and what does it cost?

The answer decides whether the HBF-for-weights premise behind the whole line of work holds. It also decides several design choices that are currently made on other grounds: how many flash dies to provision, which cell mode to use, where error correction lives, and whether an HBM site is worth trading for a flash one. We make four contributions, and set out the work that remains as explicit research tasks.

- 1) **A capacity baseline.** We extend LLMSimulator [23] with a second memory tier, a batch-independent shared KV cache, context parallelism and a block-level flash model, reproduce H^3 to within 1%, and show that under a fixed shoreline the flash tier is over-provisioned by 17–67 $\times$ for eight of nine models (Section IV). This surplus becomes the key resource in the rest of the paper.
- 2) **The reliability wall, quantified.** We model read-disturb reclaim, temperature-dependent retention and wear concentration for the static tier, and show that read disturb dominates retention by 51–135 $\times$, that the device lasts 2–5 years without static wear levelling, and that the most energy-efficient provisioning points leave no idle flash time in which to hide reliability work (Section V).
- 3) **Two exploitable properties.** LLM inference reads its static data in an order fixed before it runs, and most HBF capacity is unused. The first makes per-block read and age accounting exact and free; the second can buy endurance by replication. Replica rotation alone recovers the wear-levelled lifetime bound without migration (Section VI).
- 4) **Steward,** a base-die reliability manager built on these properties, and an evaluation plan covering device characterisation, thermal coupling, accuracy under bit errors, and end-to-end serving (Sections VII and VIII).

II. BACKGROUND

A. High-Bandwidth Flash

An HBF stack places NAND core dies on a base die that holds the flash controller, a staging buffer and the host interface. Bandwidth comes from parallelism across many planes and dies. Published estimates for a stack range over 0.35–1.6TB/s and 192GB–1TB, with array read latency between 1 and about 28 μ s (Table II). The OCP specification exposes 64B host reads backed by 4KiB NAND pages and requires writes to fill whole pages [12], [21]. Most studies assume SLC cells with about 10^5 program/erase (P/E) cycles; at least one assumes TLC at roughly 3×10^3 [12].

B. NAND failure mechanisms that do not need writes

Read disturb. Reading a page applies a pass voltage to every other wordline of the block, which slowly injects charge into unselected cells. Errors accumulate with the number of reads to the block, and the standard remedy is to reprogram the

block once a read count threshold is reached, consuming a P/E cycle [39].

Retention. Charge leaks from cells over time. The rate is strongly thermally activated and is commonly modelled with an Arrhenius law; HBFsim uses an activation energy of 1.04 eV taken from 3D-NAND characterisation [13], [41]. Data must be rewritten before its error rate exceeds what ECC can correct.

Error correction and read retry. Between refreshes, the raw bit-error rate (RBER) grows. Stronger codes and soft-decision decoding cost area, power and latency; when a read fails to decode, the controller retries with shifted read reference voltages, multiplying latency [40]. At SSD bandwidths these costs are hidden behind a block interface; at HBF bandwidths they sit on the path of every decode step.

III. WHAT THE LITERATURE ASSUMES

We read thirty-eight papers on HBF, flash-based inference and heterogeneous LLM memory (Section X). Table I records, for the HBM+HBF proposals, which reliability effects their evaluations model, and Table II the device parameters they assume.

The pattern is consistent. Endurance is analysed as a function of write volume, and the disagreements between papers are about write volume: with similar KV write rates near 1GB/s per GPU, one study concludes that endurance is sufficient by assuming 10^6 P/E cycles [8] and another projects 0.645 years at 10^5 [10]. Write amplification is usually taken as 1.02, a figure that traces to a host-side SMART counter rather than a NAND-level measurement [29]. The data that the whole line of work puts in HBF because it is *safe*, weights and shared prefixes, is the data whose reliability is not analysed. The only detailed treatment of read disturb for LLM weights is on-device, at batch one and room temperature, where Lincoln shows that read reclaim is affordable only with SLC, relaxed retention and wear levelling [26]. FLINT applies a similar model to an HBF weight tier and reports a lifetime of about 29 days at 10^5 P/E and 50 tokens/s, but does not model temperature, error correction or the effect of provisioning [9].

IV. METHODOLOGY AND CAPACITY BASELINE

A. Simulator

We extend LLMSimulator [23], an analytical roofline simulator that other HBF studies also build on [5], [6], [10]. Each operation takes $t = \max(F/\Pi, t_{\text{mem}})$. We split every operation's bytes by tensor tag into B_{HBM} and B_{HBF} (weights and shared cache to flash) and compute

$$t_{\text{mem}} = \begin{cases} \max\left(\frac{B_{\text{HBM}}}{\beta_{\text{HBM}}}, \frac{B_{\text{HBF}}}{\beta_{\text{HBF}}}\right) & \text{partitioned,} \\ \max\left(\frac{B_{\text{HBM}} + B_{\text{HBF}}}{\beta_{\text{link}}}, \frac{B_{\text{HBF}}}{\beta_{\text{HBF}}}\right) & \text{pooled.} \end{cases} \quad (1)$$

We added a batch-independent shared KV tensor, a second capacity budget, context parallelism across nodes with a corrected hierarchical all-reduce, configurable HBM/HBF sites

TABLE I

HBM+HBF PROPOSALS AND THE RELIABILITY EFFECTS THEIR EVALUATIONS MODEL. ✓ = MODELLED QUANTITATIVELY; ○ = DISCUSSED, ASSUMED, OR IMPLEMENTED WITHOUT RESULTS; – = ABSENT.

Work	HBF attachment	Static data in HBF	Write wear	GC / WAF	Wear levelling	Read disturb	Retention / temp.	ECC / retry cost	Accuracy under errors
H ³ [1]	behind HBM	weights, shared KV	○	○	○	–	–	–	–
Pooling [2]	behind HBM	– (KV only)	–	–	○	–	–	○	–
HMA [3]	beside HBM	experts	–	–	–	–	–	–	–
Hot-cold [4]	behind HBM	– (idle KV)	✓	○	–	–	–	–	–
Exploring [5]	replaces HBM	weights, KV	✓	–	–	–	○	–	–
KV endurance [6]	replaces HBM	weights, KV	✓	○	○	–	–	–	–
FlashAccel [8]	behind / replaces	weights, KV	✓	–	–	–	○	–	–
FLINT [9]	behind HBM	weights	–	–	–	✓	–	–	–
DASH [10]	direct + via HBM	weights	✓	○	–	–	–	–	–
HBFFlex [11]	HBF only	weights, KV	✓	✓	–	–	–	–	–
HBF Sucks? [12]	fixed budget	weights, KV	✓	○	–	–	○	–	–
HBFSim [13]	simulator	weights	–	–	–	○	○	–	–
SPLASH [17]	behind HBM	evicted KV	○	–	–	–	–	–	–
This work	behind HBM	weights, shared KV	✓	✓	✓	✓ (model)	✓ (model)	task	task

TABLE II
HBF PARAMETERS ASSUMED IN RECENT WORK.

Parameter	Range	Examples
Read latency t_R	1–28 μ s	1 [5], 3 [6], 15 [2], 20 [1], 25 [4]
Program latency	50–300 μ s	100 [2], [5], [6]
Read BW / stack	0.35–1.6 TB/s	0.35 [5], 1 [1], 1.6 [6]
Capacity / stack	192 GB–1 TB	384 [1], 512 [5], 1024 [3]
Power / stack	30–160 W	30 [16], <80 [5], 160 [1]
Endurance	3.3×10^3 – 10^6 P/E	[12], 10^5 [6], 10^6 [8]
Read-disturb limit	10^3 – 10^6 reads	[9], [26]
Junction limit	80–105 °C	80 [12], 105 [13]

TABLE III
RESOURCES PER GPU UNDER AN EIGHT-SITE SHORELINE.

Arrangement	Sites	HBM	Flash	Link	TDP
Cascaded (H ³)	8	192 GB	3072 GB	pooled	2280 W
Shared-base	8	96 GB	1536 GB	pooled	1480 W
Side-by-side	4+4	96 GB	1536 GB	partitioned	1480 W
HBF-only	1+7	24 GB	2688 GB	partitioned	1840 W

and dies per stack against a shoreline budget, and a block-level flash model (4 KiB pages, 1 MiB blocks, DRAM write combining, greedy collection, per-block erase counts). The changes are about 900 lines of C++. Following H³, we model B200 GPUs with eight 24 GB, 1 TB/s HBM3e cubes and eight 384 GB, 1 TB/s HBF cubes, Llama-3.1-405B with FP8 weights and FP16 KV, 1K input and output tokens, and 32 GPUs across four servers at ten million tokens. The model reproduces H³’s batch ratio ($2.71\times$ vs. $2.6\times$), throughput ($1.261\times$ vs. $1.25\times$) and throughput per watt ($0.553\times$ vs. $0.548\times$) at one million tokens, with only the batch ratio used for calibration.

B. Where the flash should go

The capacity study determines the operating point that the reliability analysis must work at, so we report it in some detail. With eight shoreline sites and eight dies per stack (an HBM die holds 3 GB, a flash die 48 GB), the four arrangements (Fig. 1) provide the resources in Table III.

TABLE IV
FLASH RESIDENCY PER DEVICE (10M TOKENS, 32 DEVICES).

Model	Weights	KV/token	Flash/device
Mixtral	93 GB	131 KB	45.9 GB
Llama-4 Scout	216 GB	197 KB	71.2 GB
Llama-4 Maverick	801 GB	197 KB	89.5 GB
Grok-1	633 GB	262 KB	105.7 GB
OpenMoE	66 GB	393 KB	130.9 GB
Llama-7B-MoE	13 GB	524 KB	172.2 GB
GLaM	288 GB	524 KB	180.8 GB
Llama-3.1-405B	406 GB	516 KB	181.8 GB
DeepSeek-V3	709 GB	3932 KB	1310.7 GB

The cascade is fastest at every context length we tested, and the reason is HBM capacity rather than flash bandwidth. At its own maximum batch it beats shared-base by $2.45\times$; pinning both to the same request count isolates the wiring, which contributes $1.22\times$, leaving $2.01\times$ from capacity (1457 admitted requests against 724). Re-running the cascade with half its flash still leaves it 23% ahead. At matched capacity, bandwidth and power, a pooled site link beats a partitioned one by 0.7% when flash is 70% of capacity and by 31.7% when it is 99% (Fig. 2), because a fixed split cannot lend bandwidth from an idle tier to a busy one. HBF-only packages fail on concurrency, not speed: at a pinned 175 requests they beat both shared-base and side-by-side, but 24 GB of HBM caps them at 174 concurrent requests. Yin et al. reach the same qualitative ordering [18].

C. How much flash is needed

Flash holds only weights and the shared cache, and both follow from the model definition (Table IV). Eight of nine models need at most 182 GB per device at ten million tokens with 32-way parallelism, against 3072 GB provisioned: a surplus of 17 – $67\times$. Only DeepSeek-V3, whose latent attention with 128 KV heads gives a per-token footprint 7.6 times Llama’s, needs more. For five of the models flash is a prerequisite rather than an optimisation: their shared cache alone exceeds a GPU’s 192 GB of HBM by 1.3 – $2.6\times$.

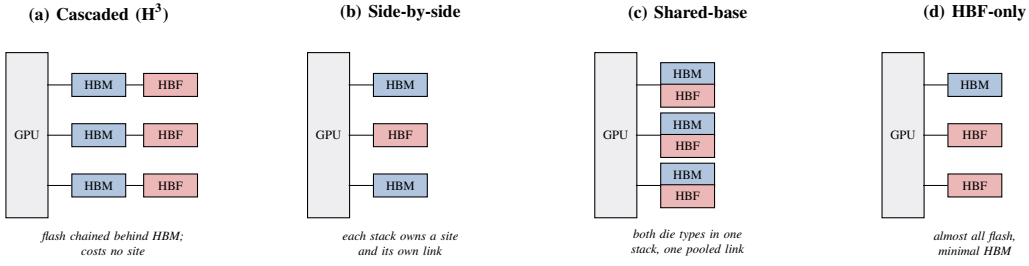


Fig. 1. The four arrangements compared under an eight-site shoreline. Only the cascaded arrangement adds flash without giving up an HBM site.

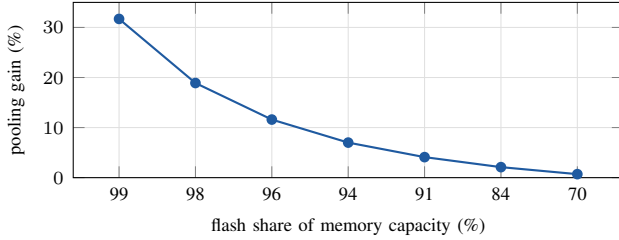


Fig. 2. Throughput gain of a pooled site link over a partitioned one at matched capacity, bandwidth and power. Our pooled model charges nothing for arbitration, so the direction of this result is built in; only its magnitude is measured.

D. Mixed-die stacks

If the chained stack’s capacity is unused, some of its dies can be DRAM. We keep the front stack at eight HBM dies and write $xHyF$ for x HBM and y flash dies per site; the site link is unchanged (Table V). Concurrency rises linearly with HBM dies, because private KV is the only consumer of HBM once static data has moved to flash, and from 8H8F to 14H2F it rises by exactly $1.75\times$ for every model we ran. Throughput falls slowly while the pooled link binds and quickly once the flash media binds, from 12H4F onwards, because a constant 165.8 GB must be read from fewer dies each step. Six of eight models prefer 12H4F for energy efficiency; Llama-3.1-405B and DeepSeek-V3 prefer 14H2F.

E. The efficiency and GPU-count consequences are conditional

The TDP column uses H^3 ’s assumption of 160 W per HBF cube. Other work assumes less than 80 W [5], [7] or budgets 30 W per stack [16]. Because throughput in our model does not depend on power, the same throughputs can be re-divided by the TDP each assumption implies (Fig. 3). 12H4F beats 8H8F on tokens per joule only if an HBF cube draws more than about 52 W; the thresholds for 10H6F and 14H2F are 43 W and 75 W. Removing flash dies also raises the minimum GPU count: 8H8F holds the static data on two GPUs, as H^3 claims, but 12H4F needs four and 14H2F eight. The concurrency gain holds regardless of power.

F. What this means for reliability

Two columns of Table V matter for the rest of the paper. C/S is the idle *capacity* available for reliability, and media

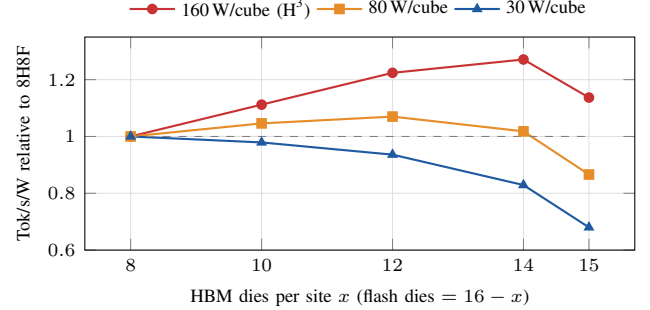


Fig. 3. Energy efficiency of mixed-die stacks under three HBF power assumptions. Throughput is unchanged; only the TDP differs.

utilisation the idle flash *time*. The designs that are most attractive for concurrency and energy have neither. This is the tension the rest of the paper examines.

V. THE RELIABILITY WALL

A. Read pressure on static data

A decode step streams the whole static footprint once. At the step times of Table V, each static block is read in $2.0\text{--}5.3 \times 10^5$ steps per day, and $3.7\text{--}9.6 \times 10^8$ steps over five years. Published read-disturb limits range from 10^3 to 10^6 reads per block [9], [26]. Every static block therefore crosses its read-disturb limit hundreds to about a million times in its service life, and must be rewritten each time. Batching does not help: weights are read once per step regardless of how many requests share the step.

B. A reclaim model

Let T be the step time, k the page reads a block receives per step, R the read-disturb threshold, τ_{ret} the retention window, S the static footprint and C the flash capacity. A block must be reclaimed every

$$\tau = \min\left(\frac{RT}{k}, \tau_{\text{ret}}(\Theta)\right), \quad (2)$$

where Θ is the die temperature. If reclaim writes land in the static region, lifetime is $L_0 = N_{\text{PE}} \tau$; if a static wear leveller spreads them over the device, $L_{\text{WL}} = N_{\text{PE}} \tau C/S$.

With FLINT-like parameters ($R = 10^6$ page reads per block, $k = 256$ pages per 1 MiB block, 10^5 P/E), τ is 11–28 minutes across the designs of Table V. The refresh write traffic is small,

TABLE V
MIXED-DIE CASCADED STACKS, LLAMA-3.1-405B, 10M TOKENS, 32 GPUS. T = REQUESTS / TOKENS/S. C/S = FLASH CAPACITY OVER THE 165.8 GB STATIC FOOTPRINT. MEDIA UTIL. = FRACTION OF EACH STEP THE FLASH MEDIA IS BUSY UNDER EQ. (1). MIN. GPUS = SMALLEST COUNT WHOSE FLASH HOLDS THE 5.82 TB STATIC FOOTPRINT.

Design	HBM	Flash	Requests	Tok/s	Tok/s/W	TDP	T (ms)	C/S	Media util.	Min. GPUs
8H8F	192 GB	3072 GB	1457	8881	0.1218	2280 W	164	18.5	64%	2
10H6F	240 GB	2304 GB	1824	8839	0.1354	2040 W	206	13.9	78%	3
12H4F	288 GB	1536 GB	2191	8579	0.1489	1800 W	255	9.3	100%	4
14H2F	336 GB	768 GB	2557	7721	0.1547	1560 W	331	4.6	100%	8
15H1F	360 GB	384 GB	2741	6380	0.1384	1440 W	430	2.3	100%	16

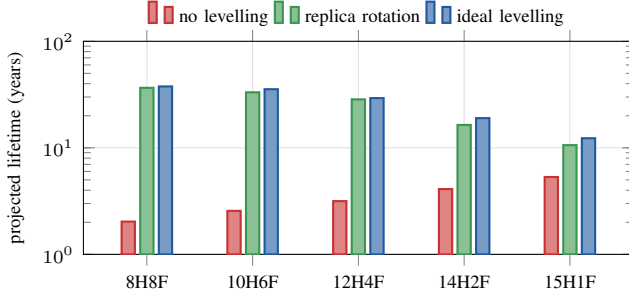


Fig. 4. Lifetime of the static tier under read-disturb reclaim alone ($R = 10^6$, $k = 256$, 10^5 P/E). Replica rotation (Section VI) reaches the ideal levelling bound without migrating data.

99–259 MB/s per device, so this is not a bandwidth problem. It is an endurance problem: L_0 is 2.0–5.3 years, and only the slowest design reaches a five-year service target (Fig. 4). With ideal static wear levelling, L_{WL} is 12–38 years. The ordering of designs reverses between the two: without levelling, $L_0 \propto T$ and removing flash dies helps because each step is slower; with levelling, $L_{WL} \propto TC$ and removing flash dies hurts.

C. Temperature

The OCP retention bound of 24 hours applies at 85 °C. With the 1.04 eV activation energy used by HBFsim [13], [41], the same data retains for about 9.6 hours at 95 °C and 4.0 hours at 105 °C, the junction limit HBFsim assumes; at 55 °C it would retain for three weeks (Fig. 5). Under the FLINT-like parameters read disturb still sets τ at every temperature in this range, by a factor of 9–22 at 105 °C and 51–135 at 85 °C across the designs. Two things change the picture, and neither has been studied. First, read disturb itself is temperature dependent, so R is a function of Θ . Second, an HBF stack is not isothermal: dies closer to the base die and to the GPU run hotter, so different dies of one stack age at different rates and a single refresh policy is wrong for most of them.

D. Wear concentration

The static region is not the only one that wears. When HBM is over-subscribed and private KV spills to flash, the spill region cycles continuously while the static region, absent reclaim, never does. In our block-level model, the spill region of 8H8F lasts 0.5 years without static wear levelling and 5.7 years with it, a factor of 10.7; the factor is 5.0 for 12H4F

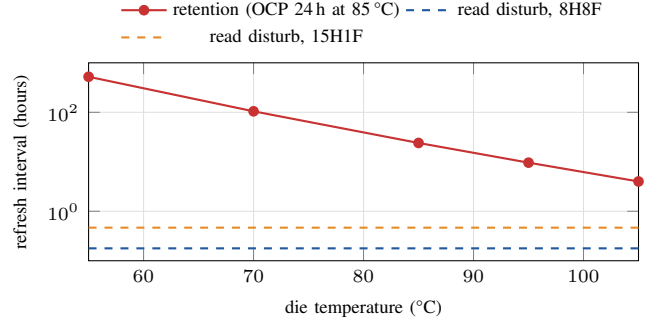


Fig. 5. Refresh interval set by retention (Arrhenius, $E_a = 1.04$ eV) and by read disturb ($R = 10^6$, $k = 256$, taken as temperature-independent here). The temperature dependence of read disturb is one of the open questions.

and 2.2 for 14H2F, because less spare capacity is available to spread the wear. KV garbage collection itself is nearly free under per-request allocation (victim blocks are fully invalid, WAF 1.33 from buffer flushes alone), because a request’s KV dies all at once. Read reclaim and spill wear draw on the same P/E budget, and no study has combined them.

E. No idle time

In an SSD, refresh and retries hide in idle time. Under Eq. (1), the flash media of 8H8F is busy 64% of each decode step, which leaves room for reclaim programs and re-reads. From 12H4F onwards the media is the bottleneck and is busy for the entire step (Table V). Any reliability work then comes directly out of decode throughput, and every read retry extends the step. The provisioning choices that the capacity study favours are exactly the ones that remove the slack reliability management needs.

F. What is not yet known

The model above bounds only the refresh-driven wear. Three larger unknowns remain, and each could move the conclusion in either direction.

Research task. Error-rate model for HBF under LLM access patterns. Build an RBER model as a joint function of P/E count, reads since last program, time since program and temperature, calibrated against published 3D-NAND characterisation for SLC and pSLC-mode TLC [39]–[41]. Replace the fixed threshold R with the read count at which RBER crosses the correction limit of a chosen code, and report τ , lifetime and refresh traffic as distributions rather than points. Establish whether the 10^3 and 10^6 thresholds used in the literature are for comparable cells, page sizes and temperatures, and which applies to HBF’s shrunk, many-plane arrays.

Research task. Thermal coupling of an HBF stack to the GPU. Derive a per-die temperature map of a flash stack chained behind HBM next to a B200-class GPU under decode power, using a compact thermal model (e.g., the 3D-ICE based models of [12], [13]) and the power cases of Table II. Feed per-die temperatures into the error model to obtain per-die refresh deadlines, and quantify how much of the stack ages faster than the average. Determine whether cascaded, side-by-side and shared-base placement differ materially in flash temperature, which would add a reliability term to the topology comparison.

Research task. Error-correction and read-retry cost at HBF bandwidth. Size BCH and LDPC decoders for 1–1.6TB/s per stack on the base die (area, power, latency) across the RBER range the error model predicts, including parity overhead on the link. Model read-retry probability as RBER grows between refreshes, its effect on the latency-hiding buffer of H^3 , and the resulting tail of decode step time and TPOT. Identify the refresh interval that minimises total cost: wear from refreshing early against decode power and latency from correcting late.

VI. TWO PROPERTIES WORTH EXPLOITING

A. The access schedule is known in advance

An SSD controller must count reads per block because its workload is unpredictable. LLM decode is not. The static region is streamed in the same layer order every step, each block receives the same k page reads per step, and the step count is known to the runtime. Per-block read counts and ages are therefore a function of a single counter and a static layout table. This removes per-block read counters, makes every refresh deadline exact rather than conservative, and tells the controller when each block will next be read, so a reclaim can be scheduled immediately after the last read before its deadline and the fresh copy used from the next step on. The same knowledge predicts the threshold-voltage drift of each block, which allows read reference voltages to be preset from age and read count instead of discovered through retries [40].

B. Most of the capacity is idle

Section IV showed that C/S ranges from 2.3 to 18.5. Storing $r \leq \lfloor C/S \rfloor$ copies of the static data and rotating which copy each step reads divides the read rate per physical block by r , so τ and L_0 grow by r . Replica rotation moves no data after the initial write, yet reaches 97% of the ideal wear-levelled lifetime for 8H8F and 12H4F and 86% for 14H2F (Fig. 4). The same replicas spread read load across plane groups, which HBFlex already does for hot prefixes to balance bandwidth [11], and give a second copy to fall back on when a read fails to decode, turning a read retry into a read from another replica. This reframes the over-provisioning result: capacity that is waste for performance is a reliability budget. It also changes the die-split question, because every flash die removed to gain concurrency removes replicas.

Research task. Capacity-for-reliability trade space. Formulate the allocation of spare HBF capacity among replicas, pSLC-mode conversion of TLC dies (lower density, several times the endurance and lower read latency), spill space for over-subscribed KV and stronger ECC parity, as an optimisation with lifetime, throughput and GPU count as objectives. Solve it per model and context length, and determine whether the answer changes the recommended die split of Section IV. Include the DeepSeek-V3 case, where the flash tier is not over-provisioned and the budget is near zero.

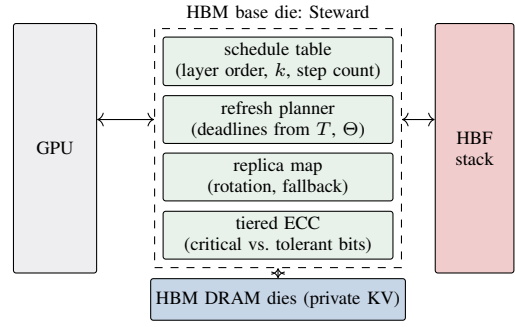


Fig. 6. Steward on the base die of a cascaded HBM stack.

VII. STEWARD: A BASE-DIE RELIABILITY MANAGER

Steward is the host-side counterpart that the OCP specification assumes but does not define. It sits on the HBM base die of a cascaded stack, between the GPU link and the downstream HBF link, next to the address router and latency-hiding buffer that H^3 already places there (Fig. 6).

Schedule table and refresh planner. The runtime registers the static layout (which blocks each layer reads, in which order) and signals step boundaries. The planner derives each block’s read count and age, combines them with the per-die temperature, and issues reclaims just after a block’s last read before its deadline, preferring steps in which the flash media has slack. Where the media has no slack (12H4F and below), it places reclaims where they delay the fewest subsequent reads.

Replica map. Static data is written r times at load. The map rotates the replica each step reads, retires replicas as they approach their deadline and redirects reads to a sibling when decoding fails.

Tiered protection. Not every bit of a weight matters equally: sign and exponent bits of FP8 and BF16 values, and the outlier channels that quantisation work treats specially, are far more sensitive than low mantissa bits. Steward applies a strong code to the sensitive portion and a light code to the rest, reducing decoder power at terabytes per second, and allows tolerant data to run closer to its correction limit before refresh.

Research task. Design and evaluate Steward end to end. Specify the schedule table, planner, replica map and tiered codes in enough detail to estimate base-die area and power against the budget left after the H^3 latency-hiding buffer (about 6.7% of a 121 mm² die). Implement them in the simulator with the error and thermal models of Section V, and compare lifetime, throughput, TPOT tail and energy against (a) no management, (b) conventional SSD-style per-block counters with threshold reclaim, and (c) FLINT’s refresh scheme [9]. Quantify how much each mechanism contributes.

Research task. Accuracy under raw bit errors. Inject bit errors into weights and shared KV of representative models (dense, MoE and latent attention; FP8 and BF16) at the RBERs the error model predicts between refreshes, and measure accuracy on long-context and reasoning benchmarks. Determine which bits, tensors and layers are critical, whether errors in the shared KV cache behave differently from errors in weights, and what residual error rate is acceptable after tiered correction. This defines how much protection and how much refresh Steward can safely remove.

VIII. EVALUATION PLAN

The research tasks above produce models; this section states how they will be combined into an answer to the problem statement.

Research task. Reliability-aware provisioning study. Repeat the capacity study of Section IV with lifetime as a constraint: for each model, context length and service target (3 and 5 years), find the die split, cell mode, replica count and GPU count that maximise throughput per dollar. Report where the reliability constraint changes the answer relative to a performance-only study, including whether side-by-side or HBF-only packages become preferable once temperature and slack are counted.

Research task. Workload realism. Replace the single input/output length with real traces (conversational, long-document and agentic sessions) and heavy-tailed request lengths; include prefill, corpus updates to the shared cache (which create genuine overwrite traffic), model swaps and multi-tenant serving, all of which change the read and write pressure on the static tier.

Research task. Validation beyond the analytical model. Check the timing consequences of refresh and retries against a cycle-level HBF model (HBF-Sim [14]) and the refresh and thermal behaviour against HBFSim [13]; compare with DASH [10], which extends the same base simulator. Where possible, measure read-disturb and retention behaviour on commercial SLC or low-latency NAND at elevated temperature to calibrate the error model.

Research task. Adversarial read pressure. In multi-tenant deployments one tenant controls part of the access stream. Analyse whether concentrated reads can raise the error rate of another tenant's weights faster than the refresh plan assumes, in the spirit of disturbance studies on DRAM and 3D NAND, and whether the schedule-based planner can detect and bound it.

IX. DISCUSSION

Why this matters beyond HBF. The read-only premise is attractive because it seems to remove the hard part of flash. If it does not hold at HBF's operating point, the consequences reach every proposal that puts weights or prefixes in flash, and the choice of what to store in HBF becomes a reliability decision as much as a performance one. If Steward-style management makes it hold cheaply, it gives the specification's host-managed refresh contract a concrete implementation.

Limitations of the current analysis. Our lifetimes use a fixed read threshold and an Arrhenius retention model with one activation energy; they bound the problem rather than predict it. The capacity study is analytical and ignores latency, which matters once retries appear. The specification details we rely on are taken from their descriptions in other papers and must be confirmed against the document itself [21].

X. RELATED WORK

HBM with HBF. H³ introduced read-only placement behind HBM [1]. Later designs add custom base dies for KV staging [2], hardware expert migration [3], hot-cold session tiering [4], burst-coalescing and refresh hiding for weights [9], direct and relayed paths [10], HBF-only packages with lifetime-aware reclamation [11], sparse attention over HBF-resident KV [17], expert replication and multi-model residency [18], and appliance-level provisioning of DRAM and host bandwidth [19]. Characterisation studies examine HBF as main memory [5], KV endurance through read-to-write ratios [6], write-aware admission for recommendation [7] and full-stack costs [12]. Simulators [13], [14], layout [15] and

other applications [16] complete the picture. Ma and Patterson list HBF among four directions for inference hardware [20].

Flash for LLMs. LLM in a Flash, AiF, Lincoln and Cambricon-LLM run single-batch inference from on-device flash [24]–[27]; Lincoln's read-reclaim analysis is the closest precedent for ours. ZnG placed Z-NAND next to GPU cores [28]. Storage-side systems offload KV or computation to SSDs [29]–[32].

NAND reliability. Read disturb, retention and their mitigation are well characterised for SSDs [39]–[42]. What is new here is the operating point, a static region read several times per second at GPU temperatures, and a host that knows its access schedule exactly.

Other heterogeneous memory. H2M2, LIA, CENT and Stratum tier LLM data across byte-addressable, write-tolerant memories [33]–[36]; RoMe and InfiniGen address access granularity and KV prefetch [37], [38].

XI. CONCLUSION

HBF promises to replace dozens of GPUs bought for capacity with a flash tier that holds read-only data. Our analysis indicates that “read-only” is not a reliability property at HBF's operating point: every static block is read two to six orders of magnitude more often than its read-disturb limit over a service life, the retention window shrinks to hours at GPU temperatures, and the provisioning points that look best for performance leave no time or capacity to manage either. The same workload that creates the problem also offers a way out, because its access schedule is known and most of its flash capacity is idle. The work set out here, an error model, thermal coupling, correction cost, accuracy under errors and the Steward manager built on them, is what it would take to know whether HBF can hold LLM state reliably, and at what price.

ACKNOWLEDGMENT

We thank the SNU SCALE lab for releasing LLMSimulator. Generative AI tools were used to help organise the literature survey and to edit the language of this manuscript. The simulator extensions, experiments, analysis and conclusions are the authors' own, and the authors take full responsibility for the content.

REFERENCES

- [1] M. Ha, E. Kim, and H. Kim, “H³: Hybrid architecture using high bandwidth memory and high bandwidth flash for cost-efficient LLM inference,” *IEEE Comput. Archit. Lett.*, vol. 25, no. 1, pp. 49–52, 2026, doi: 10.1109/LCA.2026.3660969.
- [2] J. Park, H. An, H. Suh, Y. Yoon, H. Lee, and J. Kim, “HBM-HBF-centric memory pooling architecture with custom base die for terabyte-scale LLM inference,” *IEEE Comput. Archit. Lett.*, vol. 25, no. 2, pp. 259–262, 2026, doi: 10.1109/LCA.2026.3703982.
- [3] H. Atassi, N. Zilberman, and A. Awad, “Hardware-managed heterogeneous high-bandwidth memory and flash in LLM inference systems,” Univ. of Oxford Research Archive, 2026. [Online]. Available: <https://ora.ox.ac.uk/objects/uuid:a9f80041-f2e-4714-9bc4-51c513604d36> [verify: venue]
- [4] J. Baek, W. Ji, S. Yoo, and J.-Y. Kim, “Hot-cold tiering of HBM and high bandwidth flash for agentic LLM serving,” arXiv:2609.25782, 2026.

- [5] D. Son, Y. Park, H. Cho, H. Ham, O. Mutlu, S. Lee, G. Kim, and J. Park, "Exploring high-bandwidth flash for modern LLM inference: Opportunities and challenges," *IEEE Comput. Archit. Lett.*, vol. 25, no. 2, pp. 251–254, 2026, doi: 10.1109/LCA.2026.3705817.
- [6] K. Kyung, Y. J. Moon, J. Cho, and J. H. Ahn, "High-bandwidth flash for KV caches: Endurance and performance implications," *IEEE Comput. Archit. Lett.*, vol. 25, no. 1, pp. 210–213, 2026, doi: 10.1109/LCA.2026.3695938.
- [7] D. Peng, K. Wu, T. Zuo, P. Xia, and H. Zang, "Enabling high-bandwidth flash for generative recommendation serving with write-aware KV cache policy," arXiv:2609.07175, 2026.
- [8] X. Wang, Y. Xue, X. Sun, X. Zhang, C. Dou, X. Li, and X. Chen, "FlashAccel: Leveraging high-bandwidth flash for high-throughput LLM inference," arXiv:2607.10186, 2026.
- [9] G. F. Oliveira, A. Tavakkol, X. Zhu, A. C. Yüzügüler, V. Arulchelvan, L. Cavigelli, R. Andri, M. Sadrosadati, X. Jia, O. Mutlu, K. Zhang, S. Bergman, and J. Zhang, "FLINT: Efficiently leveraging high bandwidth flash for capacity-scalable LLM inference acceleration," arXiv:2608.25062, 2026.
- [10] S. Kim, J. Jin, and J.-Y. Kim, "Beyond capacity: Scalable MoE LLM inference via high-bandwidth flash with direct GPU and HBM paths," arXiv:2608.14333, 2026.
- [11] S. Zhong, W. Xu, Y. Zhou, T. Zhao, T. Zhao, Y. Kang, C. Chang, S. Li, G. Sun, and M. Li, "HBFlex: A flexible memory system for bridging fine-grained LLM states and coarse-grained HBF parallel execution," arXiv:2609.18675, 2026.
- [12] Z. Li, Z. Bian, X. Huang, Y. Zhao, G. Sun, and Y. Zhuo, "HBF sucks? A full-stack characterization of high-bandwidth flash for KV-centric LLM serving," arXiv:2608.11668, 2026.
- [13] Y. Hu, Y. Yang, Y. Zhu, K. Chu, Y. Zheng, A. Quinn, and W. Zhang, "HBFsSim: Fast and faithful simulation of high-bandwidth flash under real GPU execution," arXiv:2609.09800, 2026.
- [14] Y. Li, J. Wang, J. Wang, L. Yang, H. Yan, X. Chai, and L. Shi, "HBF-Sim: An extensible HBF simulator for large-scale GPU memory systems," arXiv:2609.29246, 2026.
- [15] J. H. Ju, E. Chung, H. Taneja, A. Saxena, S. Jeong, H. Kim, and M. K. Qureshi, "TileLens: Efficiently using large-granularity memory systems with transparent two-dimensional memory layout," arXiv:2607.04031, 2026.
- [16] P.-K. Hsu, W. Xu, Q. Liu, T. Rosing, and S. Yu, "HAVEN: High-bandwidth flash augmented vector engine for large-scale approximate nearest-neighbor search acceleration," arXiv:2603.01175, 2026.
- [17] A. A. Jonnalagadda, A. Haputhanthri, P. Dangi, R. Juneja, W. Yue, A. Bagchi, B. Gao, T. Mitra *et al.*, "SPLASH: Co-designing sparse attention with high-bandwidth flash for efficient long-context inference," arXiv:2609.23816, 2026. [verify: complete author list]
- [18] Y. Yin, Y. Zhao, Z. Yun, G. Wu, F. Zhu, K. Tao, S. Li, F. Huang, Z. Zhang, S. Li, and H. Zheng, "Potential applications of HBF in LLM serving systems," arXiv:2608.13127, 2026.
- [19] P. Xia, T. Hao, S. Li, J. Chen, S. Wei, W. Zou, R. Zhang, and H. Zang, "Trillion-parameter MoE in a box: Decoupling memory provisioning with high-bandwidth flash," arXiv:2609.15636, 2026.
- [20] X. Ma and D. Patterson, "Challenges and research directions for large language model inference hardware," arXiv:2601.05047, 2026.
- [21] Open Compute Project, "High bandwidth flash high-level base die specification," Tech. Rep., version 0.7.0, Aug. 2026. [verify: full URL; confirm the refresh and retention clauses in the document itself]
- [22] Sandisk, "Sandisk HBF fact sheet," 2025. [Online]. Available: https://documents.sandisk.com/content/dam/asset-library/en_us/assets/public/sandisk/collateral/company/Sandisk-HBF-Fact-Sheet.pdf
- [23] S. Yun *et al.*, "Duplex: A device for large language models with mixture of experts, grouped query attention, and continuous batching," in *Proc. 57th IEEE/ACM Int. Symp. Microarchitecture (MICRO)*, 2024. [verify: author list and pages]
- [24] K. Alizadeh, S. I. Mirzadeh, D. Belenko, S. K. Khatamifard, M. Cho, C. C. Del Mundo, M. Rastegari, and M. Farajtabar, "LLM in a flash: Efficient large language model inference with limited memory," in *Proc. 62nd Annu. Meeting Assoc. Comput. Linguistics (ACL)*, 2024, pp. 12562–12584.
- [25] J. Lee, H. Kim, S. Oh, M. Chun, M. Kim, and J. Kim, "AiF: Accelerating on-device LLM inference using in-flash processing," in *Proc. 52nd Annu. Int. Symp. Comput. Archit. (ISCA)*, 2025, pp. 529–543, doi: 10.1145/3695053.3731073.
- [26] W. Sun, M. Gao, Z. Li, A. Zhang, I. Y. Chou, J. Zhu, S. Wei, and L. Liu, "Lincoln: Real-time 50–100B LLM inference on consumer devices with LPDDR-interfaced, compute-enabled flash memory," in *Proc. IEEE Int. Symp. High-Perform. Comput. Archit. (HPCA)*, 2025, pp. 1734–1750, doi: 10.1109/HPCA61900.2025.00128.
- [27] Z. Yu, S. Liang, T. Ma, Y. Cai, Z. Nan, D. Huang, X. Song, Y. Hao, J. Zhang, T. Zhi, Y. Zhao, Z. Du, X. Hu, Q. Guo, and T. Chen, "Cambricon-LLM: A chiplet-based hybrid architecture for on-device inference of 70B LLM," in *Proc. 57th IEEE/ACM Int. Symp. Microarchitecture (MICRO)*, 2024. [verify: pages and DOI]
- [28] J. Zhang and M. Jung, "ZnG: Architecting GPU multi-processors with new flash for scalable data analysis," in *Proc. ACM/IEEE 47th Annu. Int. Symp. Comput. Archit. (ISCA)*, 2020, pp. 1064–1075.
- [29] I. Jeong, S. Woo, S. Namkung, and D. Jeon, "HiFC: High-efficiency flash-based KV cache swapping for scaling LLM inference," in *Proc. 39th Conf. Neural Inf. Process. Syst. (NeurIPS)*, 2025.
- [30] X. Pan, E. Li, Q. Li, S. Liang, Y. Shan, K. Zhou, Y. Luo, X. Wang, and J. Zhang, "InstAttention: In-storage attention offloading for cost-effective long-context LLM inference," in *Proc. IEEE Int. Symp. High-Perform. Comput. Archit. (HPCA)*, 2025, pp. 1510–1525, doi: 10.1109/HPCA61900.2025.00113.
- [31] H. Jang, J. Song, C. Shin, S. U. Noh, J. Jung, J. Park, and J. Lee, "A cost-effective near-storage processing solution for offline inference of long-context LLMs," in *Proc. 31st ACM Int. Conf. Archit. Support Program. Lang. Oper. Syst. (ASPLOS)*, 2026, doi: 10.1145/3779212.3790119.
- [32] H. Jang, J. Song, J. Jung, J. Park, Y. Kim, and J. Lee, "Smart-Infinity: Fast large language model training using near-storage processing on a real system," in *Proc. IEEE Int. Symp. High-Perform. Comput. Archit. (HPCA)*, 2024.
- [33] S. Hwang, J. Kim, S. Lee, H. Kim, and J. Huh, "Hardware-based heterogeneous memory management for large language model inference," arXiv:2504.14893, 2025.
- [34] H. Kim, N. Wang, Q. Xia, J. Huang, A. Yazdanbakhsh, and N. S. Kim, "LIA: A single-GPU LLM inference acceleration with cooperative AMX-enabled CPU-GPU computation and CXL offloading," in *Proc. 52nd Annu. Int. Symp. Comput. Archit. (ISCA)*, 2025, pp. 544–558, doi: 10.1145/3695053.3731092.
- [35] Y. Gu, A. Khadem, S. Umesh, N. Liang, X. Servot, O. Mutlu, R. Iyer, and R. Das, "PIM is all you need: A CXL-enabled GPU-free system for large language model inference," in *Proc. 30th ACM Int. Conf. Archit. Support Program. Lang. Oper. Syst. (ASPLOS)*, 2025, doi: 10.1145/3676641.3716267.
- [36] Y. Pan, Z. Xia, P.-K. Hsu, L. Hu, H. Kim, J. Sharda, M. Zhou, N. S. Kim, S. Yu, T. Rosing, and M. Kang, "Stratum: System-hardware co-design with tiered monolithic 3D-stackable DRAM for efficient MoE serving," in *Proc. 58th IEEE/ACM Int. Symp. Microarchitecture (MICRO)*, 2025, doi: 10.1145/3725843.3756043.
- [37] H. Nam, S. Baek, J. Kim, M. J. Kim, and J. H. Ahn, "RoMe: Row granularity access memory system for large language models," in *Proc. IEEE Int. Symp. High-Perform. Comput. Archit. (HPCA)*, 2026.
- [38] W. Lee, J. Lee, J. Seo, and J. Sim, "InfiniGen: Efficient generative inference of large language models with dynamic KV cache management," in *Proc. 18th USENIX Symp. Oper. Syst. Design Implement. (OSDI)*, 2024, pp. 155–172.
- [39] Y. Cai, Y. Luo, S. Ghose, and O. Mutlu, "Read disturb errors in MLC NAND flash memory: Characterization, mitigation, and recovery," in *Proc. 45th Annu. IEEE/IFIP Int. Conf. Dependable Syst. Netw. (DSN)*, 2015.
- [40] Y. Cai, S. Ghose, E. F. Haratsch, Y. Luo, and O. Mutlu, "Error characterization, mitigation, and recovery in flash-memory-based solid-state drives," *Proc. IEEE*, vol. 105, no. 9, pp. 1666–1704, 2017.
- [41] Y. Luo, S. Ghose, Y. Cai, E. F. Haratsch, and O. Mutlu, "HeatWatch: Improving 3D NAND flash memory device reliability by exploiting self-recovery and temperature awareness," in *Proc. IEEE Int. Symp. High-Perform. Comput. Archit. (HPCA)*, 2018.
- [42] R.-S. Liu, C.-L. Yang, and W. Wu, "Optimizing NAND flash-based SSDs via retention relaxation," in *Proc. 10th USENIX Conf. File Storage Technol. (FAST)*, 2012.